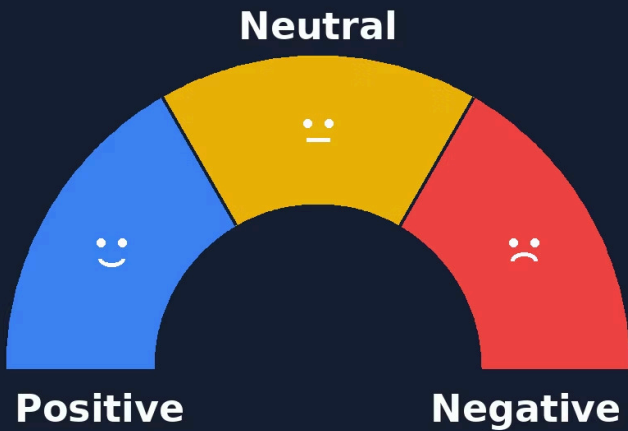
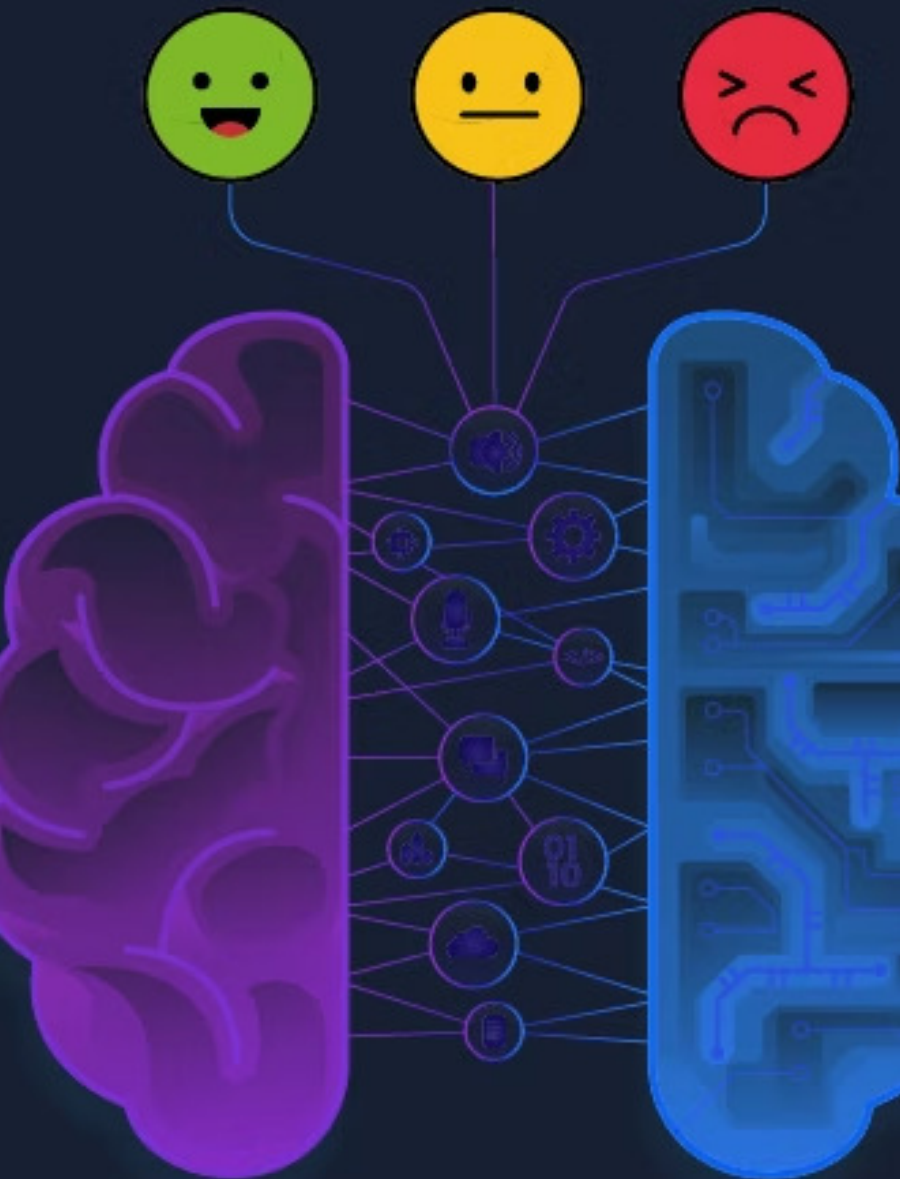


Sentiment Analysis: Preprocessing, Feature Extraction, and Modeling



Attendance: "Sentiment Analysis"





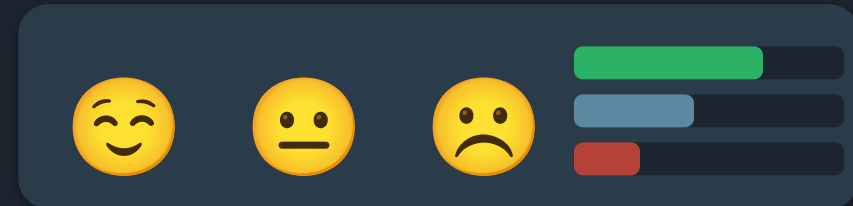
What is Sentiment Analysis?

Sentiment Analysis is a natural language processing (NLP) technique used to determine the emotional tone and subjective content within text. It classifies opinions, attitudes, and emotions as positive, negative, or neutral, and can even detect more nuanced feelings like joy or frustration.

Applications of Sentiment Analysis

Sentiment analysis is used across numerous industries to make data-driven decisions by understanding various perceptions.

- Customer Feedback Analysis
- Brand Reputation Management
- Market & Competitor Insights
- Employee Experience
- Social Media Monitoring
- Financial & Market Prediction



Types of Sentiment Analysis

Fine-Grained Sentiment Analysis

Analyzes text to classify sentiment with high specificity, distinguishing between slightly positive, positive, very positive, and similar nuanced categories.

Emotion Detection

Identifies specific human emotions expressed in text, such as joy, anger, sadness, surprise, fear, or disgust, rather than just positive or negative sentiment.

Aspect-Based Sentiment Analysis

Focuses on determining the sentiment towards specific entities or aspects within a sentence, rather than the overall sentiment of the entire text.

Multilingual Sentiment Analysis

Processes and analyzes sentiment in texts written in multiple languages, overcoming language barriers to understand global opinions and feedback.

Intent-Based Sentiment Analysis

Goes beyond emotion to recognize the user's underlying intention or goal, such as purchasing intent, cancellation intent, or a request for information.



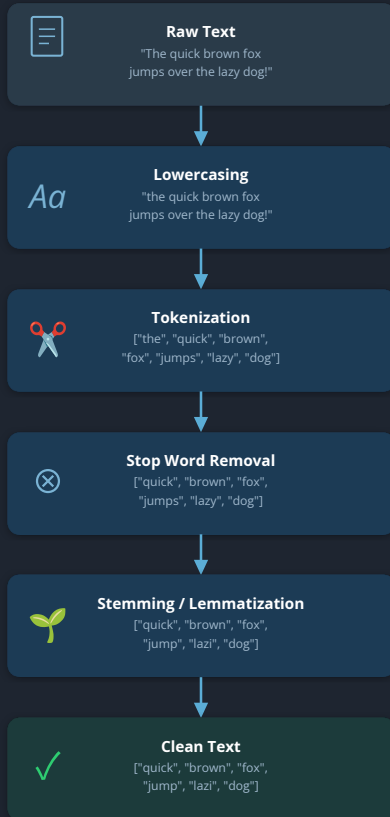
Dataset: Twitter Airline Sentiment

This dataset compiles tweets directed at major US airlines, labeled with specific sentiment categories.

- Contains ~14,000 tweets from airline customers.
- Each tweet is assigned one of three sentiment labels: **positive**, **neutral**, or **negative**.
- Recall that this would be fine-grained sentiment analysis.
- Features realistic social media text, including mentions, hashtags, and slang.

Text Preprocessing

TEXT PREPROCESSING



Why Preprocess Text?

Reduce noise

Remove artifacts (URLs, mentions, repeated characters) so models learn meaningful patterns.

Normalize

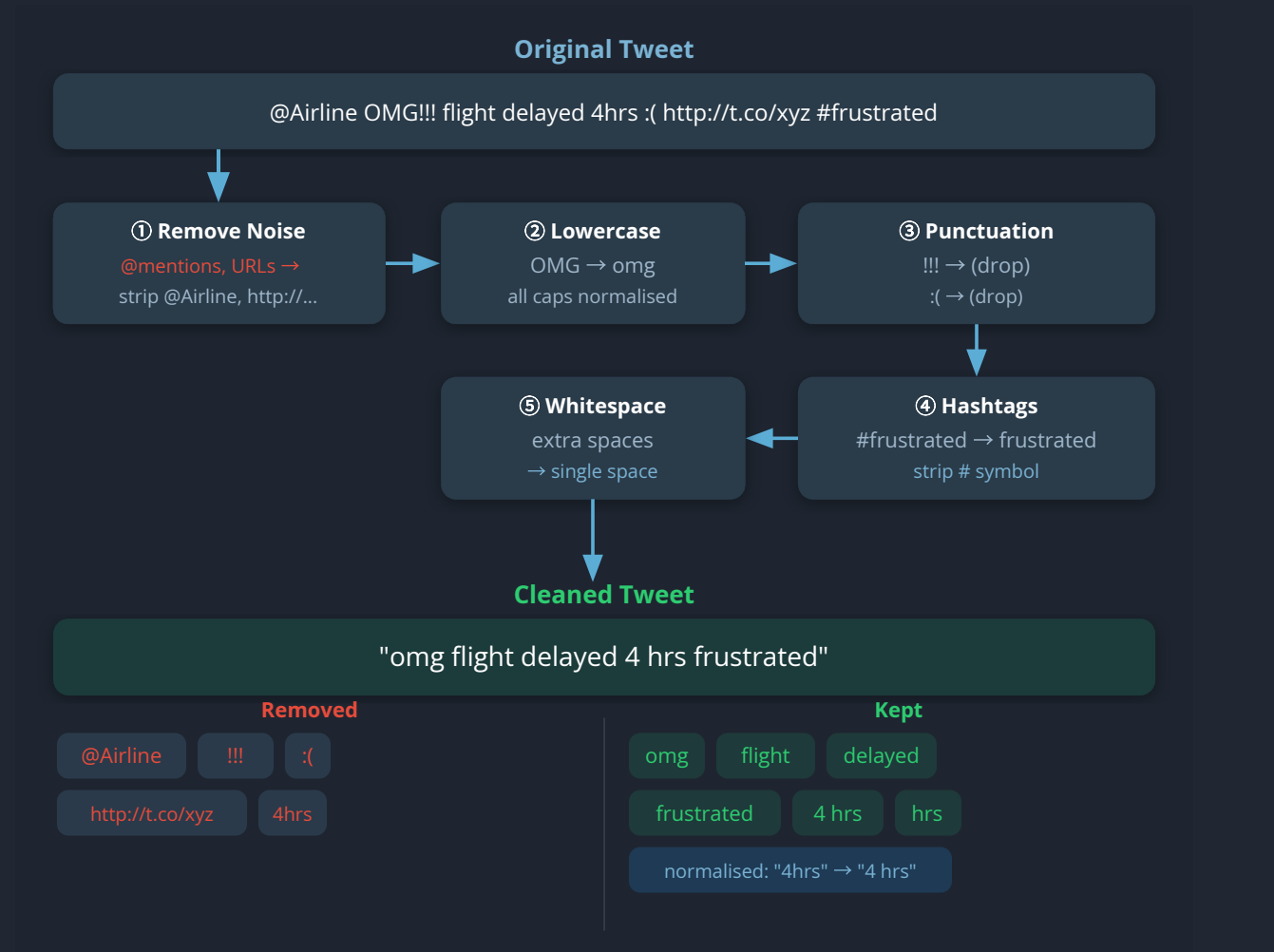
Bring tokens to a consistent form (lowercase, base words) to reduce vocabulary size.

Improve features

Cleaner input yields better feature vectors for downstream tasks like sentiment labeling.

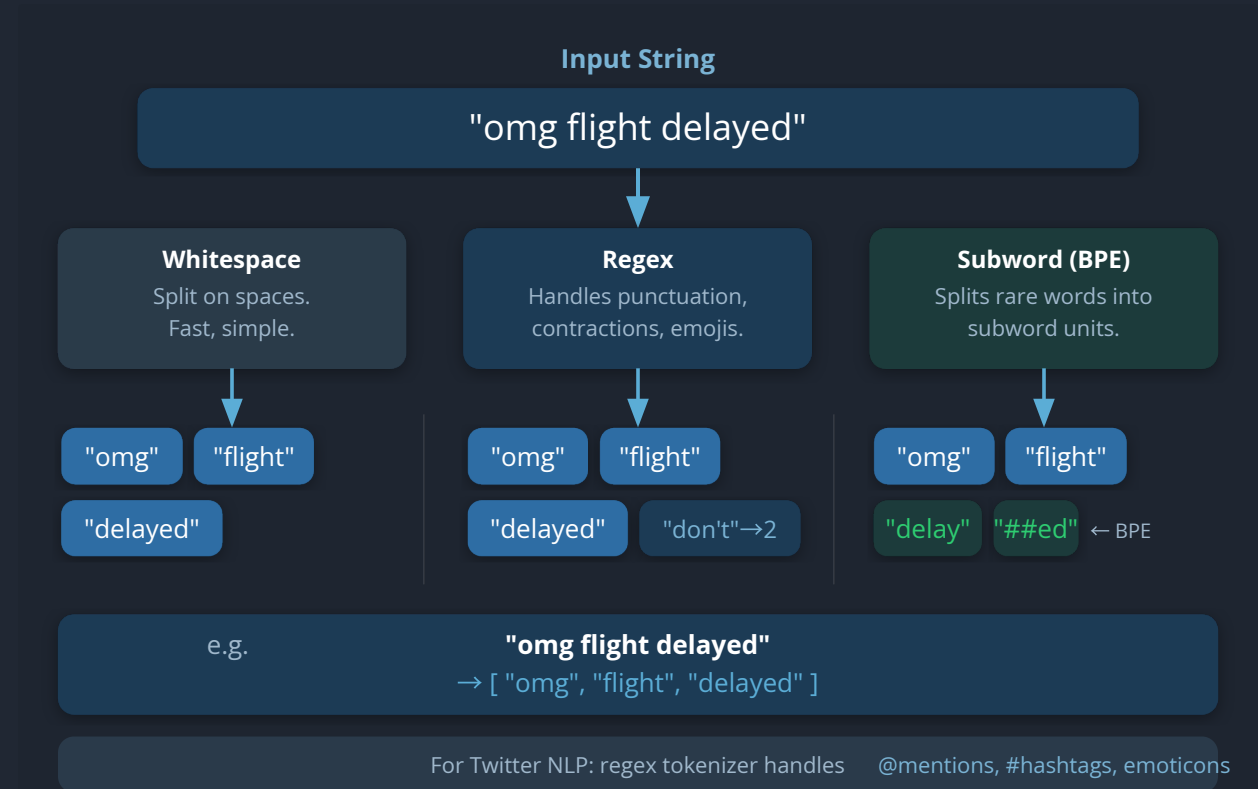
Text Cleaning

Remove or normalize noise in raw text (URLs, HTML, mentions, emojis, punctuation, extra whitespace).



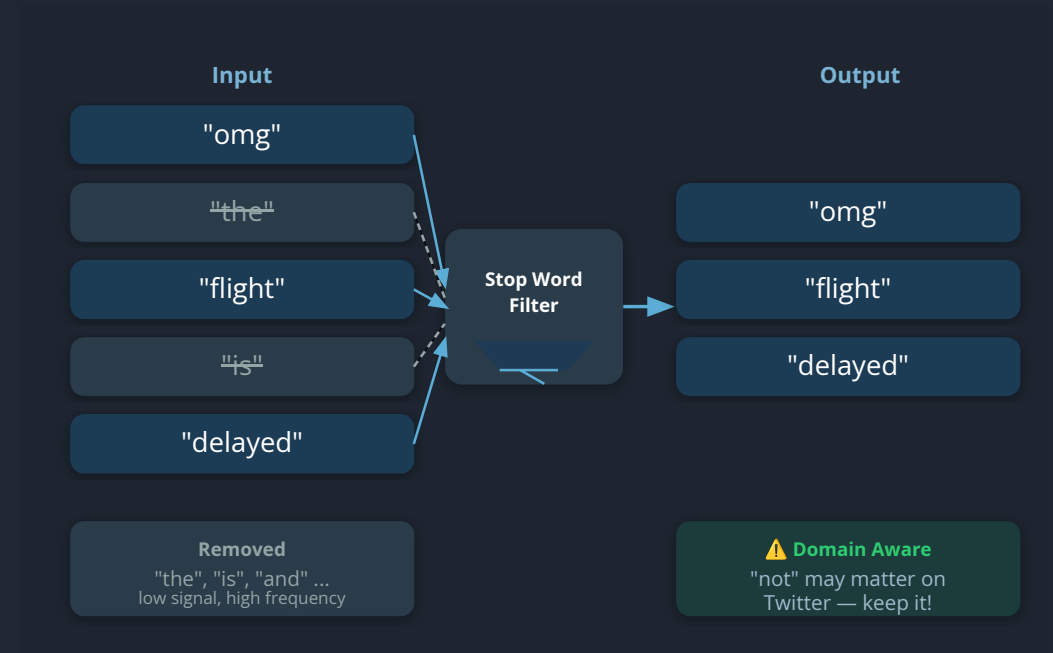
Tokenization

Split cleaned text into atomic units (tokens), typically words or subwords. Choose tokenizer by task: simple whitespace, regex, or subword (BPE/WordPiece).



Stop Word Removal

Remove very common words that carry little sentiment signal (e.g., 'the', 'is', 'and'). Use domain-aware lists: some stop words may be informative in Twitter (e.g., 'not').



Stemming & Lemmatization

Stemming

Heuristic chopping of word endings to produce a stem. Fast, can be aggressive.

Lemmatization

Uses vocabulary and POS to return the canonical lemma. More accurate but needs POS tags.

Stemming

Heuristic · Fast · Aggressive



⚠️ "studies" → "studi" — not a real word, but fast & consistent

Stemming

Fast · No POS needed · Rougher

Lemmatization

Vocabulary + POS · Accurate · Slower



✓ "studies" → "study" — real word, uses POS context to get it right

Lemmatization

Slower · Needs POS · Precise

Google Colab

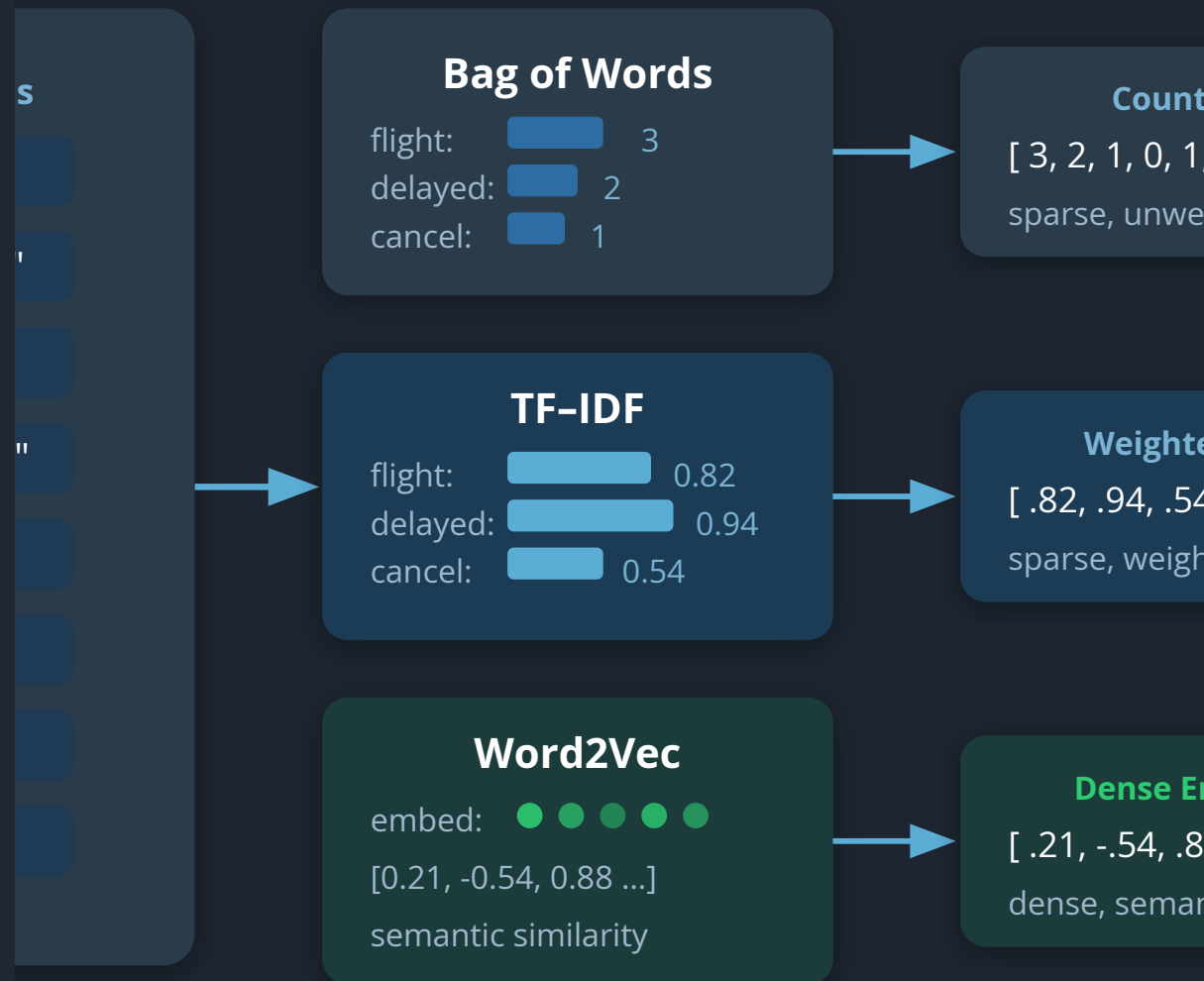


Feature Extraction: Overview

Turn token lists into numeric vectors. Three common approaches for classical NLP labs:

- Bag of Words (BoW)
- TF-IDF (weighted BoW)
- Word embeddings (Word2Vec)

We will compute and compare these before modeling.



tokens → numeric vectors → model

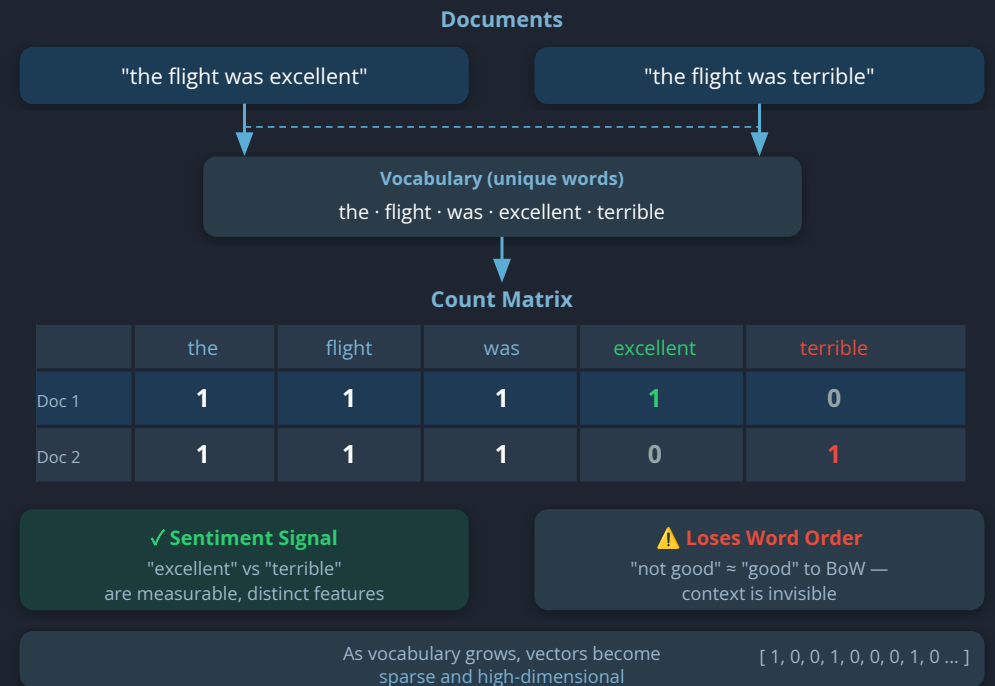
Bag of Words (BoW)

Bag of Words converts text into numerical vectors by counting word occurrences, ignoring grammar and word order.

A vocabulary of unique words is built, and each document becomes a fixed-length vector where each position is a word's count or presence.

For sentiment analysis, word frequencies (e.g., "excellent" vs. "terrible") serve as measurable signals for the model.

BoW loses context and word order (so "not good" resembles "good") and produces sparse, high-dimensional vectors as vocabulary grows.



TF-IDF

Term Frequency $\times$ Inverse Document Frequency (TF-IDF) downweights common tokens across corpus (train set) and highlights informative words for a document (single training example).

Example: A word that appears frequently in one tweet but rarely across tweets gets a higher TF-IDF score.

Practical tip: normalize vectors (L2) and experiment with n-grams (1-2) to capture short phrases like "flight delayed".



Word Embeddings

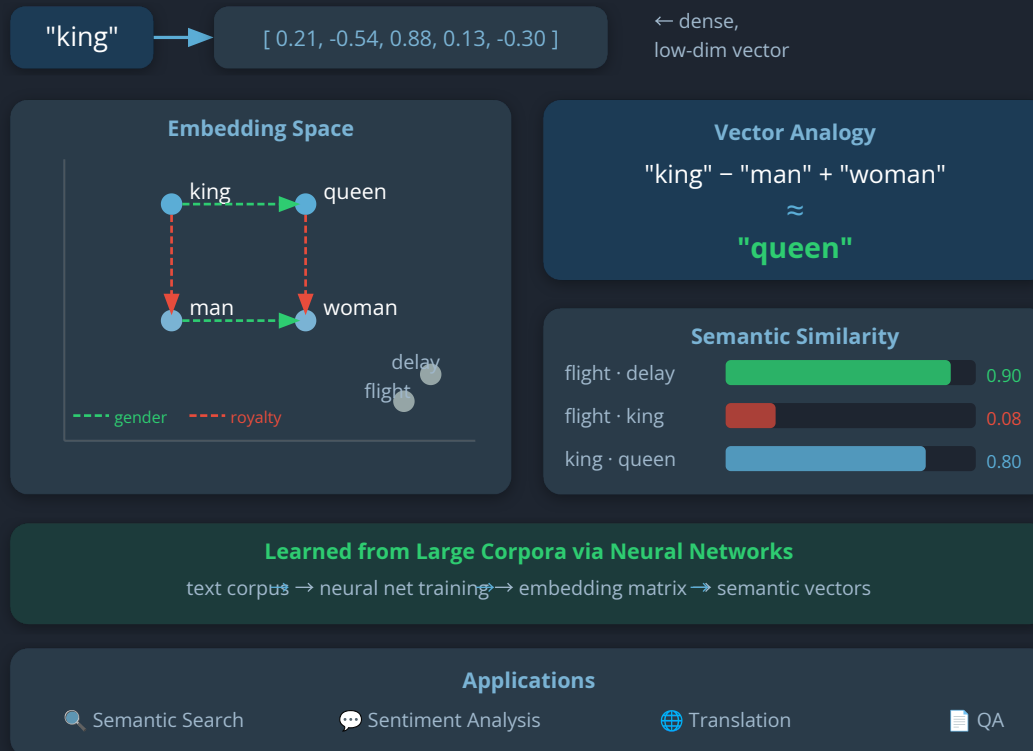
Word embeddings represent words as dense vectors, capturing semantic meaning and contextual relationships from text data.

Embeddings are typically learned from large text corpora using neural networks.

Vectors with a high dot product are semantically similar, indicating close meaning.

A classic example is the vector analogy: 'king' - 'man' + 'woman' ≈ 'queen'.

This demonstrates how embeddings encode complex semantic patterns, enabling powerful applications like semantic search.



Bag of Words vs TF-IDF vs Word Embeddings

Method	Pros	Cons
Bag of Words	Simple Interpretable Fast	Ignores word order High dimensionality Treats all words equally
TF-IDF	Downweights common words Highlights informative terms Still interpretable	Ignores word order and context Requires full corpus knowledge
Word Embeddings	Captures semantic meaning Contextual relationships Dense vectors, powerful for deep learning	Requires large training corpus Less interpretable Computationally expensive

Approaches to Sentiment Modeling

Rule-Based

Neural Networks

Machine Learning

Hybrid

Rule-Based: VADER

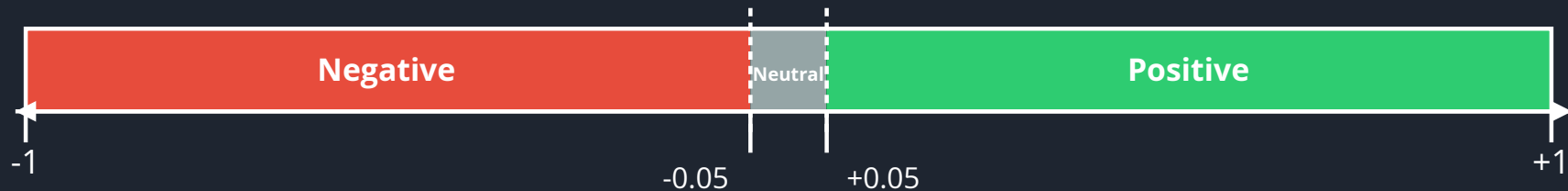
VADER (Valence Aware Dictionary and sEntiment Reasoner) uses hand-crafted sentiment dictionaries and linguistic rules, no training required. It's purpose-built for social media text and returns a **compound score** from **-1** (most negative) to **+1** (most positive).

How the score is interpreted

VADER's compound score summarizes the overall sentiment of a text.

Thresholding rules

- **compound ≥ 0.05** : Positive
- **compound ≤ -0.05** : Negative
- **Otherwise**: Neutral

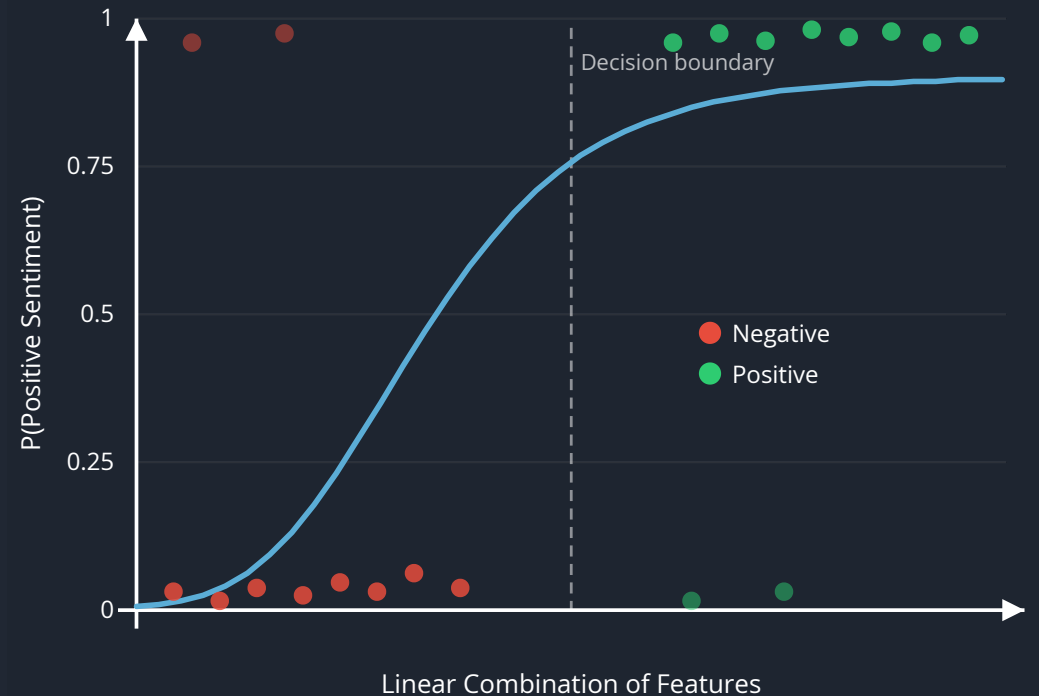


Machine Learning Models

Learn patterns from labeled training data rather than hand-crafted rules.

Logistic Regression: fits a linear decision boundary between classes, works well on sparse TF-IDF features

Naive Bayes: assumes each word is independent given the class; fast and effective on text

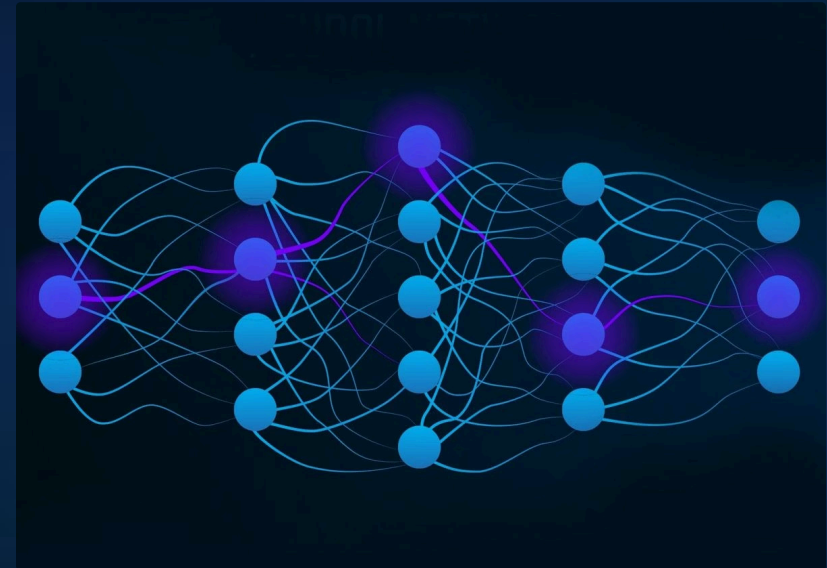


Neural Networks

Learn non-linear relationships between features and labels. More expressive than linear models but require more data and tuning.

Multilayer Perceptron (MLP):

Composed of fully connected layers with non-linear activation functions. Better suited for dense input vectors like Word2Vec than sparse TF-IDF.

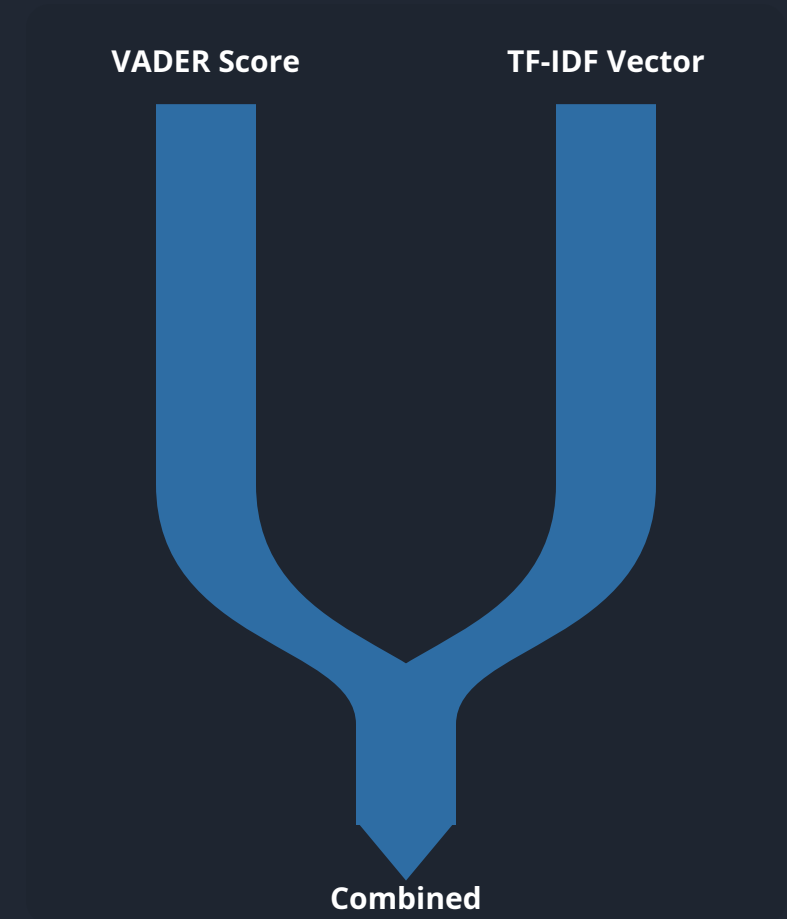


Hybrid Models

Combine signals from multiple approaches to leverage the strengths of each.

Our approach:

- Concatenate VADER compound score with TF-IDF features
- Train Logistic Regression on the combined feature vector
- The model uses both lexicon knowledge and learned word statistics simultaneously



Model Evaluation and Comparison

Accuracy alone is misleading with imbalanced classes.

- **Precision:** of all predicted positives, how many were correct?
- **Recall:** of all actual positives, how many did we catch?
- **F1 Score:** harmonic mean of precision and recall
- **Confusion Matrix:** shows exactly where a model makes mistakes

		Predicted Values	
		Positive	Negative
Actual Values	Positive	70	30 Type 2 Error
	Negative	20 Type 1 Error	180

Thank you for coming!

No meetings 3/8 and 3/15 - enjoy spring break!

Next meeting - 3/22